

PRACTICAL NO 2

AIM: Implement the Recursive Best-First Search algorithm for the same problem.

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# 1. NETWORK DEFINITION
```

```
# -----
```

```
# Road distances g(n)
```

```
graph = {
```

```
    'Mumbai Airport (CSMIA)': {
```

```
        'Sahar Road': 2,
```

```
        'Vakola': 4
```

```
    },
```

```
    'Sahar Road': {
```

```
        'Kalina': 5
```

```
    },
```

```
    'Vakola': {
```

```
        'Bandra Kurla Complex': 6
```

```
    },
```

```
    'Kalina': {
```

```
        'Kurla West': 4
```

```
    },
```

```
'Bandra Kurla Complex': {  
    'Kurla West': 4,  
    'LTT Railway Station': 7  
},
```

```
'Kurla West': {  
    'LTT Railway Station': 3  
},
```

```
'LTT Railway Station': {}  
}
```

Straight-line heuristic values $h(n)$

```
heuristics = {  
    'Mumbai Airport (CSMIA)': 10,  
    'Sahar Road': 8,  
    'Vakola': 7,  
    'Kalina': 5,  
    'Bandra Kurla Complex': 6,  
    'Kurla West': 2,  
    'LTT Railway Station': 0  
}
```

Coordinates for visualization

```
coords = {  
    'Mumbai Airport (CSMIA)': (1, 8),  
    'Sahar Road': (2, 7),  
    'Vakola': (0.5, 6),  
    'Kalina': (2.5, 5),  
    'Bandra Kurla Complex': (0.5, 4),  
    'Kurla West': (3, 3),  
    'LTT Railway Station': (4.5, 1.5)  
}  
  
# -----  
# 2. RECURSIVE BEST-FIRST SEARCH (RBFS)  
# -----  
  
def rbfs_search(start, goal):  
  
    success, path, cost, _ = rbfs(  
        start,  
        goal,  
        g=0,  
        f_limit=float('inf'),  
        path=[start]  
    )  
  
    return path, cost
```

```
def rbfs(node, goal, g, f_limit, path):

    if node == goal:
        return True, path, g, g

    neighbors = graph[node]

    if not neighbors:
        return False, [], 0, float('inf')

    successors = []

    for neighbor, distance in neighbors.items():

        if neighbor not in path:

            next_g = g + distance

            next_f = max(next_g + heuristics[neighbor],
                          g + heuristics[node])

            successors.append([next_f, neighbor, next_g])

    if not successors:
        return False, [], 0, float('inf')

    while True:
```

```
successors.sort(key=lambda x: x[0])
```

```
best = successors[0]
```

```
if best[0] > f_limit:
```

```
    return False, [], 0, best[0]
```

```
alternative = successors[1][0] if len(successors) > 1 else float('inf')
```

```
success, result_path, total_g, returned_f = rbfs(
```

```
    best[1],
```

```
    goal,
```

```
    best[2],
```

```
    min(f_limit, alternative),
```

```
    path + [best[1]]
```

```
)
```

```
best[0] = returned_f
```

```
if success:
```

```
    return True, result_path, total_g, returned_f
```

```
# -----
```

```
# 3. RUN THE ALGORITHM
```

```
# -----
```

```
path, total_dist = rbfs_search(  
    'Mumbai Airport (CSMIA)',  
    'LTT Railway Station'  
)
```

```
print("Optimal RBFS Path:")  
print(" -> ".join(path))  
print("Total Distance:", total_dist, "km")
```

```
# -----
```

```
# 4. GRAPH PLOTTING
```

```
# -----
```

```
plt.figure(figsize=(9,7))
```

```
for node, neighbors in graph.items():
```

```
    x1, y1 = coords[node]
```

```
    for neighbor, dist in neighbors.items():
```

```
        x2, y2 = coords[neighbor]
```

```
        is_path = (  
            node in path and  
            neighbor in path and
```

```
path.index(neighbor) == path.index(node) + 1  
)
```

```
color = '#2ecc71' if is_path else '#bdc3c7'
```

```
width = 3 if is_path else 1.5
```

```
plt.plot(  
    [x1, x2],  
    [y1, y2],  
    color=color,  
    linewidth=width  
)
```

```
plt.text(  
    (x1+x2)/2,  
    (y1+y2)/2,  
    f"{dist} km",  
    color='red',  
    fontsize=9  
)
```

```
for node, (x, y) in coords.items():
```

```
color = '#f1c40f' if node in path else '#3498db'
```

```
plt.scatter(  
    x,
```

```
y,  
s=700,  
color=color  
)  
  
plt.text(  
    x,  
    y,  
    f"{node}\nh={heuristics[node]}",  
    ha='center',  
    va='center',  
    fontsize=8,  
    color='white',  
    fontweight='bold'  
)  
  
plt.title(  
    f"RBFS: Mumbai Airport (CSMIA) → LTT Railway Station\nTotal Distance = {total_dist} km",  
    fontsize=12,  
    fontweight='bold'  
)  
  
plt.axis('off')  
plt.show()
```


OUTPUT:

